OIDC e federação com Azure

GH-200 | Complemento 2026 | Resumo de estudo para Cledson

1. O que resolve

OpenID Connect (OIDC) permite que um job comprove sua identidade ao provedor de nuvem e obtenha credenciais temporárias. A federação estabelece confiança entre o emissor GitHub e a identidade ou role configurada no provedor. Isso evita manter uma chave de acesso de longa duração da nuvem como secret no GitHub. [1]

2. Fluxo que você precisa entender

1. O administrador configura a confiança no provedor.
2. Uma action de autenticação solicita ao GitHub um JWT OIDC.
3. O provedor valida assinatura, validade e claims.
4. Se as condições coincidirem, emite credenciais temporárias.
5. O job acessa somente os recursos autorizados para aquela identidade. [1]

3. Três credenciais diferentes

GITHUB_TOKEN: autenticação para operações no GitHub.
Token OIDC: prova da identidade do job apresentada ao provedor.
Credenciais da nuvem: resultado da troca, usado para acessar os serviços. O JWT do GitHub não é, por si só, uma chave de acesso aos recursos do Azure. [1, 3, 4]

4. A permissão que cai na prova

```
permissions:
  id-token: write # Permite solicitar o JWT OIDC.
  contents: read # Permite leitura para checkout, se necessário.
```

write não significa escrever na nuvem. Essa permissão habilita a solicitação do token; a autorização vem das políticas da nuvem. Pode ser definida no workflow ou somente no job que precisa autenticar. Declarar a permissão também não faz login sozinho: é necessário um step que solicite e troque o token. [3]

Exemplo comentado: Azure

Configure uma aplicação no Microsoft Entra ID e seu service principal, ou uma identidade gerenciada atribuída pelo usuário. Adicione a credencial federada e atribua Azure RBAC no escopo necessário. Crie no GitHub AZURE_CLIENT_ID, AZURE_TENANT_ID e AZURE_SUBSCRIPTION_ID. Esses IDs não são senhas. [3, 4]

```
name: Verificar identidade no Azure
on:
  workflow_dispatch: # Disparo manual.
jobs:
  autenticar:
    runs-on: ubuntu-latest # Runner com Azure CLI disponível.
    permissions:
      id-token: write # Autoriza solicitar o JWT OIDC.
    steps:
      - name: Login federado no Azure
        uses: azure/login@v3 # Versão major; pode receber updates.
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }} # Identidade.
          tenant-id: ${{ secrets.AZURE_TENANT_ID }} # Diretório Entra.
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
          audience: api://AzureADTokenExchange # Azure público.
      - name: Confirmar a assinatura
        run: az account show --query name --output tsv
```

O exemplo não usa client secret nem o JSON creds. azure/login solicita o JWT e autentica no Entra por federação. Depois, comandos az usam a sessão criada. Não há checkout porque nenhum arquivo do repositório é lido. Para fixar a action em produção, selecione o SHA completo de uma release revisada. [3, 4]

Condições de confiança: iss, aud e sub

iss: quem emitiu o token.
aud: destinatário esperado.
sub: identidade/contexto aceito pelo provedor. Restrinja a confiança ao repositório e contexto pretendidos; confiar em qualquer job do GitHub seria amplo demais. [2]

```
# Exemplos do formato anterior de sub:
repo:acme/app:ref:refs/heads/main
repo:acme/app:environment:producao
repo:acme/app:pull_request
```

Com environment, o subject padrão usa o contexto do ambiente. Configure suas regras de proteção e branches de deploy. Não copie uma condição de branch para um job com environment sem conferir o subject efetivo. [2, 3]

Pegadinhas e diagnóstico

Complemento 2026: conferir o subject

A referência atual informa que repositórios criados após 15/07/2026, ou que aderiram ao formato imutável, usam IDs de proprietário e repositório no sub. Exemplo: repo:acme@123/app@456:ref:refs/heads/main. Essa mudança não está disponível no GitHub Enterprise Server. Ajuste a confiança ao formato real do repositório; os exemplos antigos continuam relevantes para repositórios que os utilizam. [2]

Configuração de confiança no Entra ID

Na aplicação, abra Certificates & secrets → Federated credentials e adicione a federação GitHub Actions. Configure repositório e contexto aceito (branch, tag, pull request ou environment), issuer e audience. Confira o subject exato antes de salvar. Para identidade gerenciada atribuída pelo usuário, configure a federação nesse recurso. [3, 4]

Azure RBAC: atribua à identidade um papel no menor escopo necessário, por exemplo um grupo de recursos. A credencial federada comprova quem pode fazer login; RBAC determina o que essa identidade pode fazer. Evite atribuir Owner apenas para resolver um erro de deploy. [3]

Como interpretar falhas

Não consegue solicitar JWT: verifique id-token: write e as permissões efetivas.
Token rejeitado na troca: compare issuer, audience, subject e validade com a credencial federada do Entra.
Login passa, deploy é negado: investigue Azure RBAC, escopo e eventual permissão de dados no serviço.
Falha ao mudar branch ou environment: confira se o contexto mudou o subject esperado. [2, 3, 4]

Guarde para a prova

OIDC autentica o job e permite obter credenciais temporárias. Não concede acesso universal, não exige runner Linux e não elimina todos os secrets da aplicação. Não registre o JWT ou credenciais nos logs. Confiança e autorização são configurações separadas. Em reusable workflows, confira as permissões efetivas do chamador e a identidade/contexto avaliado. [1, 2]

Fontes oficiais consultadas em 11/10/2026

[1] GitHub Docs: Conceitos OIDC

[2] GitHub Docs: Referência OIDC e claims

[3] GitHub Docs: Configurar OIDC no Azure

[4] GitHub Docs: Azure Login - README oficial